ACCELERATED LEARNING PATH

BUILD A GPT-LIKE LANGUAGE MODEL FROM SCRATCH

A practical 2-week tutorial based on

Sebastian Raschka's book

WEEK 1

Architecture

Tokenizer -> Attention -> GPT

WEEK 2

Training

Pretrain -> Fine-tune -> Evaluate

CAPSTONE

Portfolio

Code, metrics, GitHub, blog

14 DAYS | 20-30 HOURS | COMPLETE SOURCE APPENDIX

Contents

A 14-day path from raw text to a trained and fine-tuned GPT-like model.

This is a 14-day, practice-first path through Sebastian Raschka's *Build a Large Language Model (From Scratch)*. It compresses Chapters 1-7 into about 27 hours and ends with four portfolio artifacts: a GPT implementation, a small pretrained model, a spam classifier, and an instruction-tuned model.

The goal is not to read every sentence. The goal is to understand every tensor transformation that appears in the working end-to-end pipeline.

Outcomes and pace

Week	Focus	Time	Evidence you should produce
1	Text pipeline, attention, GPT architecture	13-14 hours	Tokenizer tests, attention shape checks, untrained GPT generation
2	Pretraining, decoding, pretrained weights, fine-tuning	13-14 hours	Loss curve, GPT-2 comparison, classifier metrics, instruction responses

Days 7 and 14 are lighter integration days. If you only study six days per week, merge Day 7 into Day 6 and Day 14 into Day 13.

How to use this tutorial

For each day:

1. Read only the listed book pages (25-40 minutes).
2. Type the minimal implementation without copying (35-60 minutes).
3. Compare it with the complete module in this repository (15 minutes).
4. Run the practice task and record shapes, loss, or output (30-60 minutes).
5. Answer the five questions from memory (10 minutes).

Do not begin a training run until the relevant forward pass and loss have passed their shape checks.

Page-reference convention

The PDF does not contain stable source-line metadata. References therefore use the book's printed page followed by the physical PDF page in parentheses. For the numbered chapters, the mapping is:

> ***printed book page N = physical PDF page N + 22***

For example, tokenization is discussed on printed page 22 (PDF page 44).

Chapter	Printed pages	PDF pages	Core deliverable
1. Understanding LLMs	1-16	23-38	Mental model of pretraining and fine-tuning
2. Working with text	17-49	39-71	Tokenizer and next-token data loader
3. Attention	51-91	73-113	Causal multi-head attention
4. GPT model	92-127	114-149	Complete decoder-only GPT
5. Pretraining	128-168	150-190	Loss, training, decoding, GPT-2 weights
6. Classification	169-203	191-225	Spam classifier
7. Instructions	204-249	226-271	Instruction fine-tuning and evaluation

Setup

The core project expects Python, PyTorch, tiktoken, NumPy, and pandas. Day 11 also follows the book's original GPT-2 TensorFlow-checkpoint path and needs TensorFlow and tqdm:

```
BASH
```

```
python -m pip install torch tiktoken numpy pandas
# Day 11 only:
python -m pip install "tensorflow>=2.15" "tqdm>=4.66"
```

Run commands from this directory so local module imports resolve:

BASH

```
cd llm-from-scratch-2week
python -m py_compile config.py tokenizer.py attention.py model.py train.py finetune.py eval.py
python -m unittest discover -s tests -v
```

Choose a device once in each notebook or driver script:

PYTHON

```
import torch

if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")
```

Week 1 - Core Architecture

Day 1 - Understanding LLMs and text preparation

Timebox: 2 hours **Read:** Chapter 1, printed pp. 2-14 (PDF 24-36); Chapter 2, printed pp. 18-32 (PDF 40-54). The original tokenization discussion begins on printed p. 21 (PDF 43).

Core concepts

- An LLM estimates a conditional distribution over the next token. Repeating

next-token prediction creates text generation.

- Pretraining learns general language patterns from unlabeled text;

fine-tuning adapts the resulting foundation model to labeled classes or instruction-response behavior.

- A GPT is a decoder-only transformer: token and position embeddings enter a

stack of causally masked transformer blocks and a vocabulary projection.

Key code excerpts

Listings 2.1-2.4 are on printed pp. 22, 25, 27, and 31 (PDF 44, 47, 49, and 53). Their essential operations are:

```
# Listing 2.1
with open("the-verdict.txt", "r", encoding="utf-8") as f:
    raw_text = f.read()

# Listing 2.2
all_words = sorted(set(preprocessed))
vocab = {token: integer for integer, token in enumerate(all_words)}

# Listings 2.3-2.4
preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', text)
preprocessed = [item.strip() for item in preprocessed if item.strip()]
preprocessed = [
    item if item in self.str_to_int else "<|unk|>"
    for item in preprocessed
]
ids = [self.str_to_int[token] for token in preprocessed]
```

Important: `<|unk|>` handles unseen words in the teaching tokenizer, while `<|endoftext|>` separates documents. GPT-2's BPE tokenizer does not need an unknown token (printed pp. 29-34; PDF 51-56).

Minimal implementation (type this yourself)

```
import re

class Tokenizer:
    def __init__(self, text):
        tokens = self.split(text)
        words = sorted(set(tokens)) + ["<|endoftext|>", "<|unk|>"]
        self.to_id = {word: i for i, word in enumerate(words)}
        self.to_text = {i: word for word, i in self.to_id.items()}

    @staticmethod
    def split(text):
        parts = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        return [part.strip() for part in parts if part.strip()]

    def encode(self, text):
        tokens = [t if t in self.to_id else "<|unk|>" for t in self.split(text)]
        return [self.to_id[token] for token in tokens]

    def decode(self, ids):
        text = " ".join(self.to_text[i] for i in ids)
        return re.sub(r'\s+([,.:;?!"()\'])', r"\1", text)
```

Compare with `SimpleTokenizerV1`, `SimpleTokenizerV2`, and `build_vocabulary` in `tokenizer.py`.

Check your understanding

1. What conditional probability does a next-token language model learn?
2. Why can a neural network not consume raw strings directly?

3. What information does `<|endoftext|>` provide during pretraining?
4. Why is lowercasing all input not always harmless?
5. Why can an encode/decode round trip differ in whitespace yet preserve tokens?

Practice task

Download and load *The Verdict*, build the V2 vocabulary, and assert a round trip on a passage that occurs in the story:

```python
from tokenizer import (
    SimpleTokenizerV2,
    build_vocabulary,
    download_verdict,
    read_text,
)

path = download_verdict()
raw_text = read_text(path)
tokenizer = SimpleTokenizerV2(build_vocabulary(raw_text))
sample = raw_text[:150]
ids = tokenizer.encode(sample)
print(len(raw_text), len(ids), tokenizer.decode(ids))
```

Deliverable: `day01-notes.md` containing vocabulary size, the first 30 tokens, and one observed limitation of the simple tokenizer.

Stretch goal: Run the same code on a Project Gutenberg text and compare vocabulary size and unknown-token rate.

Day 2 - BPE tokenization and data sampling

Timebox: 2 hours **Read:** printed pp. 33-46 (PDF 55-68). BPE begins on p. 33; the sliding window on p. 35; Listings 2.5-2.6 are on pp. 37-38 (PDF 59-60); token and position embeddings are on pp. 41-46 (PDF 63-68).

Core concepts

- BPE represents frequent strings as whole tokens and unfamiliar strings as

smaller subwords or bytes; GPT-2 uses a fixed 50,257-token vocabulary.

- A sliding window turns one token stream into `(input, target)` pairs where

every target is its input shifted one position left.

- Token embeddings encode identity; learned positional embeddings inject

order because self-attention alone is permutation-agnostic.

Key code excerpts

```
tokenizer = tiktoken.get_encoding("gpt2")
token_ids = tokenizer.encode(txt)

for i in range(0, len(token_ids) - max_length, stride):
    input_chunk = token_ids[i:i + max_length]
    target_chunk = token_ids[i + 1:i + max_length + 1]
    self.input_ids.append(torch.tensor(input_chunk))
    self.target_ids.append(torch.tensor(target_chunk))

dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)
```

Minimal implementation

```
import tiktoken
import torch
from torch.utils.data import Dataset, DataLoader

class NextTokenDataset(Dataset):
    def __init__(self, text, max_length=128, stride=128):
        ids = tiktoken.get_encoding("gpt2").encode(text)
        self.inputs, self.targets = [], []
        for start in range(0, len(ids) - max_length, stride):
            window = ids[start:start + max_length + 1]
            self.inputs.append(torch.tensor(window[:-1]))
            self.targets.append(torch.tensor(window[1:]))

    def __len__(self):
        return len(self.inputs)

    def __getitem__(self, index):
        return self.inputs[index], self.targets[index]

def make_loader(text, batch_size=4, max_length=128, stride=128):
    dataset = NextTokenDataset(text, max_length, stride)
    return DataLoader(dataset, batch_size=batch_size, shuffle=True,
                      drop_last=True, num_workers=0)
```

Check your understanding

1. How does BPE encode a word that never appeared in its training corpus?
2. Why is `target_chunk` offset by exactly one token?
3. What changes when `stride < max_length`?
4. Why can heavy overlap increase overfitting on a tiny corpus?
5. Why must token and positional embeddings have the same final dimension?

Practice task

Create loaders with `(max_length, stride)` equal to `(8, 1)`, `(8, 4)`, and `(8, 8)`. Print the first two input rows and verify that targets equal inputs shifted by one. Record dataset length and overlap for each setting.

Deliverable: a three-row table comparing stride, number of examples, overlap, and expected compute.

Stretch goal: Use `torch.nn.Embedding` to add token and absolute position embeddings, then replace the learned positions with a first attempt at rotary positional encoding (RoPE).

Day 3 - Self-attention fundamentals

Timebox: 2 hours **Read:** printed pp. 52-64 (PDF 74-86), especially simple self-attention on pp. 56-60 and all-token attention on pp. 61-64.

Core concepts

- Fixed recurrent state creates a bottleneck for long dependencies;

attention gives each token a direct weighted path to relevant tokens.

- An attention score is a similarity value. Softmax turns a row of scores

into positive weights that sum to one.

- A context vector is a weighted sum of input vectors, so it has the same

embedding dimension as the values being combined.

Key code excerpts

The vectorized version on printed pp. 62-63 (PDF 84-85) is the line to internalize:

```
attn_scores = inputs @ inputs.T
attn_weights = torch.softmax(attn_scores, dim=-1)
all_context_vecs = attn_weights @ inputs
```

Minimal implementation

```
import torch

inputs = torch.tensor([
    [0.43, 0.15, 0.89],
    [0.55, 0.87, 0.66],
    [0.57, 0.85, 0.64],
    [0.22, 0.58, 0.33],
    [0.77, 0.25, 0.10],
    [0.05, 0.80, 0.55],
])

# Step-by-step for the second token.
query = inputs[1]
scores = torch.empty(inputs.shape[0])
for i, token in enumerate(inputs):
    scores[i] = torch.dot(token, query)
weights = torch.softmax(scores, dim=0)
context = torch.zeros_like(query)
for i, token in enumerate(inputs):
    context += weights[i] * token

# Vectorized for every token.
score_matrix = inputs @ inputs.T
weight_matrix = torch.softmax(score_matrix, dim=-1)
contexts = weight_matrix @ inputs

assert torch.allclose(weights.sum(), torch.tensor(1.0))
assert torch.allclose(context, contexts[1])
assert contexts.shape == inputs.shape
print(weight_matrix, contexts)
```

Check your understanding

1. What does a larger dot product mean geometrically?
2. Along which dimension must softmax operate for token-to-token attention?
3. Why does every attention row sum to one?
4. What are the shapes of scores and contexts for input `[T, d]`?
5. What crucial capability is missing from this parameter-free mechanism?

Practice task

Use the six vectors above for "Your journey starts with one step." Label the 6x6 matrix, identify the largest weight in each row, and explain one reason the pattern is not linguistically meaningful yet.

Stretch goal: Plot the attention weights as a heatmap and compare raw dot products with cosine similarity.

Day 4 - Self-attention with trainable weights

Timebox: 2 hours **Read:** printed pp. 64-73 (PDF 86-95). The step-by-step Q/K/V projection is on pp. 65-70; Listings 3.1-3.2 are on pp. 70-73.

Core concepts

- Learned query, key, and value projections let the task decide which

relationships to retrieve and what information to aggregate.

- Scores are `Q @ K^T`; values do not influence relevance, only the content

of the resulting context vector.

- Dividing by `sqrt(d_k)` prevents large dot products from pushing softmax

into a near one-hot, low-gradient regime.

Key code excerpts

```
keys = self.W_key(x)
queries = self.W_query(x)
values = self.W_value(x)
attn_scores = queries @ keys.T
attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5, dim=-1
)
context_vec = attn_weights @ values
```

`SelfAttentionV1` stores `nn.Parameter` matrices directly; V2 uses `nn.Linear`, which supplies an optimized initialization. See `attention.py`.

Minimal implementation

```
import torch
from torch import nn

class SelfAttention(nn.Module):
    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        q = self.query(x)
        k = self.key(x)
        v = self.value(x)
        scores = q @ k.transpose(-2, -1)
        weights = torch.softmax(scores / k.shape[-1]**0.5, dim=-1)
        return weights @ v

torch.manual_seed(123)
x = torch.randn(6, 3)
layer = SelfAttention(d_in=3, d_out=2)
out = layer(x)
assert out.shape == (6, 2)
out.sum().backward()
for name, parameter in layer.named_parameters():
    assert parameter.grad is not None, name
```

Check your understanding

1. Why are Q, K, and V three separate projections of the same input?
2. Why does `K` need a transpose in the score product?
3. Which dimension determines the scale factor?
4. Why do V1 and V2 differ before their weights are synchronized?
5. What shape does a `Linear(d_in, d_out)` weight use internally?

Practice task

Initialize `SelfAttentionV1` and `SelfAttentionV2`. Copy V2's transposed weights into V1, then prove their outputs match with `torch.allclose`.

Stretch goal: Add an option that returns attention weights for debugging, then write a gradient check for all Q/K/V parameters.

Day 5 - Causal and multi-head attention

Timebox: 2.5 hours **Read:** printed pp. 74-90 (PDF 96-112). Causal masking is on pp. 74-81; Listing 3.3 is p. 81; the wrapper is p. 84; efficient Listing 3.5 is pp. 86-87.

Core concepts

- A causal mask sets future-token scores to `-inf` before softmax, producing

zero probability without a second normalization pass.

- Dropout on attention weights regularizes which token-to-token connections

the model relies on during training; it is disabled in evaluation mode.

- Multi-head attention splits `d_out` into independent subspaces, performs

attention in parallel, concatenates the heads, and applies an output projection.

Key code excerpts

```
attn_scores = queries @ keys.transpose(2, 3)
mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
attn_scores.masked_fill_(mask_bool, -torch.inf)
attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5, dim=-1
)
context = (attn_weights @ values).transpose(1, 2)
context = context.contiguous().view(batch_size, num_tokens, self.d_out)
```

Minimal implementation

```
import torch
from torch import nn

class CausalMHA(nn.Module):
    def __init__(self, d_in, d_out, heads, context, dropout=0.1):
        super().__init__()
        assert d_out % heads == 0
        self.heads = heads
        self.head_dim = d_out // heads
        self.d_out = d_out
        self.q = nn.Linear(d_in, d_out, bias=False)
        self.k = nn.Linear(d_in, d_out, bias=False)
        self.v = nn.Linear(d_in, d_out, bias=False)
        self.proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask", torch.triu(torch.ones(context, context), diagonal=1)
        )

    def forward(self, x):
        batch, tokens, _ = x.shape
        def split(projection):
            return projection(x).view(
                batch, tokens, self.heads, self.head_dim
            ).transpose(1, 2)

        q, k, v = split(self.q), split(self.k), split(self.v)
        scores = q @ k.transpose(2, 3)
        scores.masked_fill_(
            self.mask[:tokens, :tokens].bool(), -torch.inf
        )
        weights = torch.softmax(scores / self.head_dim**0.5, dim=-1)
        context = (self.dropout(weights) @ v).transpose(1, 2)
        context = context.contiguous().view(batch, tokens, self.d_out)
        return self.proj(context)

x = torch.randn(2, 6, 12)
assert CausalMHA(12, 12, 3, 16)(x).shape == (2, 6, 12)
```

Check your understanding

1. Why mask scores before softmax rather than weights after softmax?
2. Why is the causal mask registered as a buffer rather than a parameter?

3. What condition must hold between `d_out` and `num_heads`?
4. Why is `.contiguous()` used after a transpose and before `.view()`?
5. Which four dimensions appear in the per-head score tensor?

Practice task

Initialize GPT-2 small attention (`d_in=d_out=768`, 12 heads, context 1024) and verify a small batch shape using only 16 input tokens. Compare parameter counts for 1, 12, and 16 heads while holding `d_out` constant.

Stretch goal: Replace learned absolute positions with RoPE applied to Q and K. Write down why V is not rotated.

Day 6 - GPT architecture components

Timebox: 2 hours **Read:** printed pp. 99-116 (PDF 121-138). Listing 4.2 is p. 103; Listings 4.3-4.4 are pp. 105 and 107; shortcut Listing 4.5 is pp. 110-111; TransformerBlock Listing 4.6 is p. 115.

Core concepts

- Layer normalization stabilizes each token across its feature dimension;

GPT-2 uses pre-normalization before attention and feed-forward sublayers.

- The feed-forward network expands each token independently to `4*d`, applies

GELU, then contracts to `d`.

- Residual connections preserve an identity path for activations and

gradients, making deep stacks trainable.

Key code excerpts

```
mean = x.mean(dim=-1, keepdim=True)
var = x.var(dim=-1, keepdim=True, unbiased=False)
norm_x = (x - mean) / torch.sqrt(var + 1e-5)

self.layers = nn.Sequential(
    nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
    GELU(),
    nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
)
```

Minimal implementation

```
from torch import nn
from attention import MultiHeadAttention
from model import FeedForward, LayerNorm

class Block(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.att = MultiHeadAttention(
            d_in=cfg["emb_dim"],
            d_out=cfg["emb_dim"],
            context_length=cfg["context_length"],
            dropout=cfg["drop_rate"],
            num_heads=cfg["n_heads"],
            qkv_bias=cfg["qkv_bias"],
        )
        self.ff = FeedForward(cfg)
        self.norm1 = LayerNorm(cfg["emb_dim"])
        self.norm2 = LayerNorm(cfg["emb_dim"])
        self.drop = nn.Dropout(cfg["drop_rate"])

    def forward(self, x):
        shortcut = x
        x = self.drop(self.att(self.norm1(x)))
        x = x + shortcut
        shortcut = x
        x = self.drop(self.ff(self.norm2(x)))
        return x + shortcut
```

Check your understanding

1. Why does LayerNorm use `dim=-1` for `[batch, tokens, embedding]`?
2. Why is `unbiased=False` used in the variance calculation?
3. What useful capacity is gained by the feed-forward 4x expansion?
4. What shape constraint allows a residual addition?
5. How does pre-norm differ from post-norm?

Practice task

Count parameters in one GPT-2 small feed-forward module and one attention module. Explain why the feed-forward module is larger. Verify that a block maps `[2, 4, 768]` to `[2, 4, 768]`.

Stretch goal: Replace GELU with SwiGLU and compare parameter count. Then try freezing attention and training only the feed-forward layers.

Day 7 - Assemble the GPT model

Timebox: 1.5 hours **Read:** printed p. 95 and pp. 117-125 (PDF 117 and 139-147). GPTModel Listing 4.7 is p. 119; greedy generation Listing 4.8 is p. 124.

Core concepts

- Token and positional embeddings are added, passed through repeated

transformer blocks, normalized, and projected to one logit per vocabulary item at every input position.

- The output shape is `[batch, tokens, vocab_size]`; only the last row is

needed to select the next autoregressive token.

- Model size is determined mostly by embedding width, block count, and the

vocabulary matrices; context length affects positional parameters and attention compute rather than most weights.

Key code excerpts - GPT-2 small configuration

```python
GPT_CONFIG_124M = {
    "vocab_size": 50257,
    "context_length": 1024,
    "emb_dim": 768,
    "n_heads": 12,
    "n_layers": 12,
    "drop_rate": 0.1,
    "qkv_bias": False,
}
```

Minimal implementation - complete model

```python
import torch
from torch import nn
from model import LayerNorm, TransformerBlock

class GPT(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.token_embedding = nn.Embedding(
            cfg["vocab_size"], cfg["emb_dim"]
        )
        self.position_embedding = nn.Embedding(
            cfg["context_length"], cfg["emb_dim"]
        )
        self.dropout = nn.Dropout(cfg["drop_rate"])
        self.blocks = nn.Sequential(*[
            TransformerBlock(cfg) for _ in range(cfg["n_layers"])
        ])
        self.norm = LayerNorm(cfg["emb_dim"])
        self.head = nn.Linear(cfg["emb_dim"], cfg["vocab_size"], bias=False)

    def forward(self, token_ids):
        _, tokens = token_ids.shape
        positions = torch.arange(tokens, device=token_ids.device)
        x = self.token_embedding(token_ids)
        x = x + self.position_embedding(positions)
        x = self.dropout(x)
        x = self.blocks(x)
        return self.head(self.norm(x))

def greedy_generate(model, ids, new_tokens, context):
    for _ in range(new_tokens):
        with torch.no_grad():
            logits = model(ids[:, -context:])[:, -1, :]
        next_id = torch.argmax(logits, dim=-1, keepdim=True)
        ids = torch.cat([ids, next_id], dim=1)
    return ids
```

The complete implementation and parameter counter are in `model.py`.

Check your understanding

1. Why are position IDs identical for every example in a batch?

2. Why does the output head return 50,257 values per token?
3. Why is greedy decoding deterministic in evaluation mode?
4. What happens when generation exceeds the context length?
5. Why did the book's untied implementation count about 163M parameters

while GPT-2 small is called 124M?

Practice task

Instantiate the tiny configuration first, run a forward pass, and generate 10 random-looking tokens. Then instantiate medium, large, and XL configs from `config.py` only if memory permits. Calculate parameter counts without running a forward pass.

Stretch goal: Tie output-head weights to token-embedding weights and confirm the parameter count drops. Add a key/value cache design sketch for faster generation.

Week 1 checkpoint

You are ready for Week 2 only if all are true:

- [ ] `decode(encode(text))` is semantically equivalent to the input.
- [ ] Every attention row sums to approximately one before dropout.
- [ ] Future positions receive zero causal attention.
- [ ] A transformer block preserves `[batch, tokens, emb_dim]`.
- [ ] GPT returns `[batch, tokens, vocab_size]` and can greedily append tokens.

Week 2 - Training and Fine-Tuning

Day 8 - Pretraining loss and evaluation

Timebox: 2 hours **Read:** printed pp. 129-145 (PDF 151-167). Listing 5.1 is pp. 131-132; text-generation loss is pp. 132-140; loader loss Listing 5.2 is p. 144.

Core concepts

- Cross-entropy measures negative log probability assigned to the correct next

token. Lower is better; a perfect prediction approaches zero.

- Flattening `[batch, tokens, vocab]` to `[batch*tokens, vocab]` aligns it

with flattened target IDs for PyTorch cross-entropy.

- Perplexity is `exp(cross_entropy)`: the effective number of plausible next

choices under the model. Compare it only across the same tokenization.

Key code excerpts

```
def text_to_token_ids(text, tokenizer):
    encoded = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
    return torch.tensor(encoded).unsqueeze(0)

def token_ids_to_text(token_ids, tokenizer):
    return tokenizer.decode(token_ids.squeeze(0).tolist())

logits = model(input_batch)
loss = torch.nn.functional.cross_entropy(
    logits.flatten(0, 1), target_batch.flatten()
)
```

Minimal implementation

```
import torch
import torch.nn.functional as F

def loss_batch(model, inputs, targets, device):
    inputs, targets = inputs.to(device), targets.to(device)
    logits = model(inputs)
    return F.cross_entropy(logits.flatten(0, 1), targets.flatten())

def loss_loader(model, loader, device, batches=None):
    if len(loader) == 0:
        return float("nan")
    batches = len(loader) if batches is None else min(batches, len(loader))
    total = 0.0
    for i, (inputs, targets) in enumerate(loader):
        if i == batches:
            break
        total += loss_batch(model, inputs, targets, device).item()
    return total / batches

def perplexity(loss):
    return torch.exp(torch.tensor(loss)).item()
```

Check your understanding

1. Why does cross-entropy accept logits rather than softmax probabilities?
2. Why must target IDs be integer class indices?
3. What does a loss of `ln(50257)` mean for this vocabulary?
4. Why disable gradients while evaluating loss?
5. Why is perplexity not directly comparable across different tokenizers?

Practice task

Use `TINY_CONFIG`, one data-loader batch, and `calc_loss_batch`. Print input, target, logits, and flattened shapes. Confirm random-model loss is near `ln(50257)`, then compute perplexity.

Stretch goal: Implement token-level loss without reduction and visualize which positions are hardest.

Day 9 - The pretraining loop

Timebox: 2.5 hours **Read:** printed pp. 146-150 (PDF 168-172). Listing 5.3 spans pp. 147-148; the AdamW discussion is on p. 148 and overfitting is visible on pp. 149-150.

Core concepts

- A training step clears stale gradients, calculates loss, backpropagates,

and updates parameters. One epoch is one pass over the training loader.

- AdamW decouples weight decay from adaptive gradient updates and is the

standard practical baseline for transformer training.

- A falling training loss with rising validation loss indicates memorization;

tiny-corpus pretraining demonstrates mechanics, not general language skill.

Key code excerpts

```
optimizer.zero_grad()
loss = calc_loss_batch(input_batch, target_batch, model, device)
loss.backward()
optimizer.step()
tokens_seen += input_batch.numel()
```

Minimal implementation

```
def train(model, train_loader, val_loader, optimizer, device,
          epochs=5, eval_every=5, eval_batches=5):
    history = {"train": [], "validation": [], "tokens": []}
    tokens_seen, step = 0, -1
    for epoch in range(epochs):
        model.train()
        for inputs, targets in train_loader:
            optimizer.zero_grad()
            loss = loss_batch(model, inputs, targets, device)
            loss.backward()
            optimizer.step()
            tokens_seen += inputs.numel()
            step += 1
            if step % eval_every == 0:
                model.eval()
                with torch.no_grad():
                    train_loss = loss_loader(
                        model, train_loader, device, eval_batches
                    )
                    val_loss = loss_loader(
                        model, val_loader, device, eval_batches
                    )
                history["train"].append(train_loss)
                history["validation"].append(val_loss)
                history["tokens"].append(tokens_seen)
                print(epoch + 1, step, train_loss, val_loss)
    return history
```

The listing-faithful version with sample generation is `train_model_simple` in `train.py`.

Check your understanding

1. What error occurs if `optimizer.zero_grad()` is omitted?
2. Why call `model.train()` and `model.eval()` explicitly?
3. What does weight decay discourage?
4. Why is validation data never used for `backward()`?
5. What evidence distinguishes learning from memorization here?

Practice task

Train on *The Verdict* for five epochs. Use `TINY_CONFIG` on CPU; use `GPT_CONFIG_124M_TRAIN` only with enough memory. Start with `lr=4e-4`, `weight_decay=0.1`, batch size 2, context 256. Save train/validation loss, tokens seen, two generated samples, and runtime.

Stretch goal: Add gradient clipping, linear warmup, and cosine decay from Appendix D. Compare the first 20 updates with the simple loop.

Day 10 - Decoding and checkpoints

Timebox: 2 hours **Read:** printed pp. 151-160 (PDF 173-182). Temperature is pp. 152-155, top-k is pp. 155-157, Listing 5.4 is pp. 157-158, and checkpointing is pp. 159-160.

Core concepts

- Temperature below 1 sharpens a distribution; above 1 flattens it. A zero

setting is treated as deterministic argmax rather than division by zero.

- Top-k masks every logit below the kth largest value to `-inf`, preventing

low-probability tail tokens from being sampled.

- Save both model and optimizer state to resume training faithfully; save only

the model state for inference.

Key code excerpts

```
if top_k is not None:
    top_logits, _ = torch.topk(logits, top_k)
    min_val = top_logits[:, -1]
    logits = torch.where(
        logits < min_val,
        torch.tensor(float("-inf")).to(logits.device),
        logits,
    )
if temperature > 0.0:
    probs = torch.softmax(logits / temperature, dim=-1)
    idx_next = torch.multinomial(probs, num_samples=1)
else:
    idx_next = torch.argmax(logits, dim=-1, keepdim=True)
```

Minimal implementation

```
def sample_generate(model, ids, new_tokens, context,
                    temperature=0.0, top_k=None, eos=None):
    for _ in range(new_tokens):
        with torch.no_grad():
            logits = model(ids[:, -context:])[:, -1, :]
        if top_k is not None:
            values, _ = torch.topk(logits, top_k)
            threshold = values[:, -1, None]
            logits = logits.masked_fill(logits < threshold, -torch.inf)
        if temperature > 0:
            probabilities = torch.softmax(logits / temperature, dim=-1)
            next_id = torch.multinomial(probabilities, 1)
        else:
            next_id = torch.argmax(logits, dim=-1, keepdim=True)
        ids = torch.cat([ids, next_id], dim=1)
        if eos is not None and torch.all(next_id == eos):
            break
    return ids
```

Save and restore:

```
torch.save({
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
}, "checkpoint.pth")
```

Check your understanding

1. Why does high temperature increase diversity and error risk together?
2. Why is `-inf` useful before softmax?
3. Which settings make generation deterministic?
4. Why should inference call `model.eval()` after loading?
5. What optimizer information is lost when only model weights are saved?

Practice task

Generate five 50-token samples for each of: greedy; `T=0.7, k=10`; `T=1.0, k=40`; and `T=1.4, k=25`. Record repetition, coherence, novelty, and seed. Save and reload the model, then confirm greedy output is identical.

Stretch goal: Add top-p (nucleus) sampling and a repetition penalty.

Day 11 - Load pretrained OpenAI GPT-2 weights

Timebox: 2 hours, excluding the one-time download **Read:** printed pp. 160-168 (PDF 182-190). The external download helper is introduced on p. 161; model-size configuration is pp. 163-164; Listing 5.5 maps weights on pp. 165-166.

Core concepts

- Architecture must match checkpoint shapes exactly: context 1024, correct

width/layers/heads, and Q/K/V biases enabled for original GPT-2.

- OpenAI stores combined Q/K/V arrays; they must be split into three and

transposed to match PyTorch `Linear` weight layout.

- GPT-2 ties output projection weights to token embeddings; copying `wte` to

both locations reproduces that checkpoint convention.

Key code excerpts

```
q_w, k_w, v_w = np.split(
    params["blocks"][b]["attn"]["c_attn"]["w"], 3, axis=-1
)
gpt.trf_blocks[b].att.W_query.weight = assign(
    gpt.trf_blocks[b].att.W_query.weight, q_w.T
)
gpt.out_head.weight = assign(gpt.out_head.weight, params["wte"])
```

Minimal implementation

The full mapping is necessarily longer than 30 lines, so Day 11's minimal implementation is the verified orchestration around `load_weights_into_gpt`:

```
import tiktoken
from train import (
    load_pretrained_gpt2,
    text_to_token_ids,
    token_ids_to_text,
)
from model import generate_text_simple

model, config, settings = load_pretrained_gpt2(
    "gpt2-small (124M)", models_dir="gpt2"
)
tokenizer = tiktoken.get_encoding("gpt2")
prompt = "Every effort moves you"
ids = generate_text_simple(
    model,
    text_to_token_ids(prompt, tokenizer),
    max_new_tokens=25,
    context_size=config["context_length"],
)
print(settings)
print(token_ids_to_text(ids, tokenizer))
```

`train.py` downloads the official `gpt_download.py` module referenced by the book, then executes the complete Listing 5.5 mapping.

Check your understanding

1. Why must `qkv_bias=True` for the original GPT-2 checkpoint?
2. Why are OpenAI Q/K/V weights split along the final axis?
3. Why are most source matrices transposed during assignment?
4. How does a shape-checking assignment helper prevent silent corruption?
5. What output behavior quickly reveals an incorrect weight mapping?

Practice task

Use the same prompt and deterministic decoder with your Day 9 checkpoint and OpenAI GPT-2 small. Compare grammar, continuation relevance, and memorization. Log model parameter count, checkpoint size, and generation time.

Stretch goal: Load medium GPT-2 if memory permits, or implement a state dictionary converter that saves the mapped model so TensorFlow is no longer needed on later runs.

Day 12 - Fine-tune for spam classification

Timebox: 2.5 hours **Read:** printed pp. 170-201 (PDF 192-223). Data Listings 6.1-6.5 are pp. 172-180; pretrained loading is p. 182; head Listing 6.7 is p. 186; accuracy Listing 6.8 is pp. 192-193; training Listing 6.10 is pp. 196-197; inference Listing 6.12 is p. 201.

Core concepts

- Classification fine-tuning predicts a fixed label set; instruction

fine-tuning remains generative and is more flexible but more expensive.

- Replace the vocabulary head with a two-class head, freeze most pretrained

weights, and train the final block, final normalization, and new head.

- The last token representation can attend to every prior token under a

causal mask, so it is used as the sequence-level classification summary.

Key code excerpts

```
for param in model.parameters():
    param.requires_grad = False
model.out_head = torch.nn.Linear(BASE_CONFIG["emb_dim"], 2)
for param in model.trf_blocks[-1].parameters():
    param.requires_grad = True
for param in model.final_norm.parameters():
    param.requires_grad = True

logits = model(input_batch)[:, -1, :]
loss = torch.nn.functional.cross_entropy(logits, target_batch)
```

Minimal implementation

```
import tiktoken
import torch
from finetune import (
    configure_classifier,
    create_spam_dataloaders,
    download_and_unzip_spam_data,
    prepare_spam_csvs,
    train_classifier_simple,
)
from train import load_pretrained_gpt2

tokenizer = tiktoken.get_encoding("gpt2")
data_path = download_and_unzip_spam_data()
csv = prepare_spam_csvs(data_path, "spam-data")
train_loader, val_loader, test_loader, max_length = create_spam_dataloaders(
    csv["train"], csv["validation"], csv["test"], tokenizer
)
model, config, _ = load_pretrained_gpt2("gpt2-small (124M)")
model = configure_classifier(model, config["emb_dim"])
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=5e-5, weight_decay=0.1
)
history = train_classifier_simple(
    model, train_loader, val_loader, optimizer, device,
    num_epochs=5, eval_freq=50, eval_iter=5,
)
torch.save({
    "model_state_dict": model.state_dict(),
    "config": config,
    "max_length": max_length,
}, "spam-classifier.pth")
```

Check your understanding

1. Why balance the ham and spam examples in this teaching setup?
2. Why must all examples in a batch have equal token length?

3. Why train on the final output token rather than every token?
4. Which parameters remain trainable after classifier setup?
5. Why is validation accuracy not a substitute for held-out test accuracy?

Practice task

Fine-tune for five epochs. Report train/validation/test accuracy, confusion counts, maximum input length, trainable parameter count, and runtime. Test at least two original messages using `classify_text` in `eval.py`.

Stretch goal: Compare last-block tuning, full-model tuning, and LoRA from Appendix E. Keep the split and random seed fixed.

Day 13 - Fine-tune for instruction following

Timebox: 2.5 hours **Read:** printed pp. 205-232 (PDF 227-254). Prompt Listing 7.2 is p. 209; InstructionDataset is p. 213; final collate Listing 7.5 is p. 220; loaders are p. 225; pretrained model Listing 7.7 is p. 227; training Listing 7.8 is p. 231.

Core concepts

- Supervised instruction tuning still uses next-token loss; the difference is

that examples are formatted as instruction, optional input, and response.

- Dynamic per-batch padding reduces wasted compute. Targets are inputs shifted

by one, and extra padding targets become `-100`, PyTorch's ignored index.

- A capable pretrained foundation is essential. Fine-tuning teaches response

behavior; it does not cheaply create broad knowledge from scratch.

Key code excerpts

```
def format_input(entry):
    instruction_text = (
        "Below is an instruction that describes a task. "
        "Write a response that appropriately completes the request."
        f"\n\n### Instruction:\n{entry['instruction']}"
    )
    input_text = (
        f"\n\n### Input:\n{entry['input']}" if entry["input"] else ""
    )
    return instruction_text + input_text

mask = targets == 50256
indices = torch.nonzero(mask).squeeze()
if indices.numel() > 1:
    targets[indices[1:]] = -100
```

Minimal implementation

```
import tiktoken
import torch
from finetune import (
    create_instruction_dataloaders,
    download_and_load_instruction_data,
    format_input,
    split_instruction_data,
)
from train import load_pretrained_gpt2, train_model_simple

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = tiktoken.get_encoding("gpt2")
data = download_and_load_instruction_data()
train_data, val_data, test_data = split_instruction_data(data)
train_loader, val_loader, test_loader = create_instruction_dataloaders(
    train_data, val_data, test_data, tokenizer,
    device=device, batch_size=8, allowed_max_length=1024,
)
model, config, _ = load_pretrained_gpt2("gpt2-medium (355M)")
model.to(device)
optimizer = torch.optim.AdamW(
    model.parameters(), lr=5e-5, weight_decay=0.1
)
train_losses, val_losses, tokens_seen = train_model_simple(
    model, train_loader, val_loader, optimizer, device,
    num_epochs=2, eval_freq=5, eval_iter=5,
    start_context=format_input(val_data[0]), tokenizer=tokenizer,
)
torch.save(model.state_dict(), "gpt2-medium355M-sft.pth")
```

Use GPT-2 small or `allowed_max_length=256` if memory is limited.

Check your understanding

1. Why append one padding token before creating shifted targets?

2. Why keep the first `50256` target but replace later ones with `-100`?
3. What does `ignore_index=-100` change in cross-entropy?
4. Why can different instruction batches have different sequence lengths?
5. What tradeoff is involved in masking instruction tokens from the loss?

Practice task

Fine-tune for two epochs, save the state dict, and record three validation prompts before and after tuning. Treat lower validation loss as a diagnostic, not proof that responses are correct.

Stretch goal: Try the Phi-3 prompt style, a larger public instruction dataset, or the LoRA approach from Appendix E. Compare quality per trainable parameter and per minute.

Day 14 - Evaluation and portfolio polish

Timebox: 2 hours **Read:** printed pp. 233-247 (PDF 255-269). Test response Listing 7.9 is p. 237; Ollama query Listing 7.10 is pp. 242-243; score Listing 7.11 is p. 246.

Core concepts

- Generative evaluation needs multiple signals: held-out loss, task-specific exact/semantic checks, human review, and cautious model-based scoring.
- Save prompt, reference response, model response, decoding settings, model checkpoint, and seed so an evaluation is reproducible.
- An evaluator LLM is not ground truth. Its rubric, nondeterminism, and own errors must be documented.

Key code excerpts

```
generated_text = token_ids_to_text(token_ids, tokenizer)
response_text = (
    generated_text[len(input_text):]
    .replace("### Response:", "")
    .strip()
)
test_data[i]["model_response"] = response_text

score = query_model(prompt, model="llama3")
scores.append(int(score))
```

Minimal implementation

```
import json
from eval import generate_model_scores, generate_test_responses, summarize_scores

responses = generate_test_responses(
    model=model,
    test_data=test_data,
    tokenizer=tokenizer,
    device=device,
    context_size=config["context_length"],
    max_new_tokens=256,
)
with open("instruction-data-with-response.json", "w") as file:
    json.dump(responses, file, indent=4)

# Optional: requires a running local Ollama server and evaluator model.
scores = generate_model_scores(
    responses,
    json_key="model_response",
    evaluator_model="llama3",
)
print(summarize_scores(scores))
```

Check your understanding

1. Why can low validation loss coexist with poor factual responses?
2. What information must be stored to reproduce a generated response?
3. Why should model-based scores be manually spot-checked?
4. Which evaluation is appropriate for spam but insufficient for open text?
5. What result would demonstrate portfolio value beyond "the code runs"?

Practice task and final capstone

Run and document this complete pipeline:

1. Download and tokenize *The Verdict*.

2. Prove the sliding-window alignment with assertions.
3. Run the tiny GPT forward pass and pretrain it for five epochs.
4. Compare its continuation with pretrained GPT-2 small.
5. Fine-tune GPT-2 small for spam classification and report test accuracy.
6. Fine-tune GPT-2 small or medium on instruction data and save 20 held-out

prompt/reference/response triples.

7. Manually score those 20 examples with a written rubric. Optionally add a

local evaluator LLM and compare its scores with yours.

8. Package one inference path behind a local CLI or a small FastAPI endpoint.

Stretch goal: Add RoPE to your from-scratch model, run instruction LoRA, or pretrain on a Project Gutenberg collection. Choose one and report the controlled comparison rather than adding all three superficially.

Common pitfalls and fixes

Shape mismatches in attention

Symptom	Likely cause	Fix
`mat1 and mat2 shapes cannot be multiplied`	Q/K/V projection or transpose is wrong	Print Q, K, V; expect `[B, H, T, D_h]` before scores
Residual addition fails	Attention or feed-forward output is not `emb_dim`	Keep block input/output `[B, T, D]`
Mask broadcasting fails	Mask exceeds or does not cover current tokens	Slice `mask[:T, :T]`
`.view()` fails after transpose	Tensor is noncontiguous	Call `.contiguous()` before `.view()`
Softmax rows do not sum to one	Wrong dimension	Use `dim=-1`, the key-token axis
Pretrained assignment fails	Config or transpose mismatch	Match model size and use the assignment shape check

Use this shape ledger while debugging:

```
token IDs                        [B, T]
embeddings                       [B, T, D]
Q, K, V before split             [B, T, D]
Q, K, V after split              [B, H, T, D/H]
attention scores/weights          [B, H, T, T]
combined context                 [B, T, D]
language-model logits             [B, T, V]
classification logits (last row)  [B, C]
```

GPU out-of-memory errors

Attention memory grows approximately with `T^2`, while activation memory also grows with batch size, layer count, and embedding width. Reduce resources in this order:

1. Lower batch size: 8 -> 4 -> 2 -> 1.
2. Lower context/allowed maximum length: 1024 -> 512 -> 256 -> 128.
3. Use GPT-2 small rather than medium for instruction tuning.
4. Accumulate gradients across smaller batches if the effective batch matters.
5. Use mixed precision on supported GPUs and verify the loss remains finite.
6. Restart the process between large model variants; deleting one Python name

does not always immediately return reserved accelerator memory.

Do not instantiate medium, large, and XL simultaneously merely to count parameters. Calculate counts analytically or instantiate one at a time.

Slow training on CPU

- Prove code with `TINY_CONFIG`, 64-128 token contexts, and one batch first.
- Treat the 124M/355M runs as optional final experiments, not prerequisites

for understanding the mechanics.

- Use nonoverlapping training windows (`stride=max_length`) on the tiny corpus.

- Keep evaluation to a few batches while training; run full evaluation once at the end.
- On Apple Silicon, try MPS but expect small numerical differences. On a CPU, context reduction usually helps more than minor Python optimizations.

Loading pretrained weights incorrectly

- Set context length to 1024 and `qkv_bias=True` before constructing GPT-2.
- Choose dimensions for exactly one of 124M, 355M, 774M, or 1558M.
- Split combined attention arrays into Q, K, V and transpose source matrices.
- Map both token embedding and output head from `wte` for GPT-2 weight tying.
- Load weights before moving the model to the accelerator, then call `model.eval()` and generate a known coherent sample.
- Day 11's official checkpoint reader needs TensorFlow even though the model itself is PyTorch. After conversion, save a PyTorch checkpoint for reuse.

Padding and evaluation mistakes

- Classification uses the final padded position consistently because all sequences share the training maximum length.
- Instruction tuning keeps the first end-of-text target and changes later pad targets to `-100`; masking every end token prevents the model from learning when to stop.
- Never choose hyperparameters using the test set.
- Record decoding settings. Comparing greedy output from one model with a high-temperature sample from another is not a controlled evaluation.

One documented correction from the PDF

Listing 6.12 on printed p. 201 (PDF 223) obtains supported context length from `model.pos_emb.weight.shape[1]`. The weight shape is `[context_length, emb_dim]`, so the runnable `classify_text` in `eval.py` uses `shape[0]`. With GPT-2 small, axis 1 is 768 and axis 0 is the correct 1024.

Listing-to-code index

Book source	Printed page (PDF page)	Complete implementation
Listings 2.1-2.4	22, 25, 27, 31 (44, 47, 49, 53)	`tokenizer.py`: reading, vocabulary, V1/V2
Listings 2.5-2.6	37-38 (59-60)	`GPTDatasetV1`, `create_dataloader_v1`
Listings 3.1-3.2	70-73 (92-95)	`SelfAttentionV1`, `SelfAttentionV2`
Listings 3.3-3.5	81, 84, 86-87 (103, 106, 108-109)	causal, wrapper, efficient MHA
Listings 4.2-4.4	103, 105, 107 (125, 127, 129)	LayerNorm, GELU, FeedForward
Listings 4.6-4.8	115, 119, 124 (137, 141, 146)	block, GPTModel, greedy generation
Listings 5.1-5.4	131-132, 144, 147-148, 157-158 (153-154, 166, 169-170, 179-180)	text conversion, loss, loop, sampling
Listing 5.5	165-166 (187-188)	`load_weights_into_gpt`
Listings 6.1-6.5	172-180 (194-202)	spam download, split, dataset, loaders
Listings 6.7-6.10	186, 192-197 (208, 214-219)	head, accuracy/loss, classifier loop
Listing 6.12	201 (223)	corrected `classify_text`
Listings 7.2-7.6	209-225 (231-247)	formatting, dataset, collate, loaders
Listings 7.7-7.9	227, 231, 237 (249, 253, 259)	loading, training orchestration, responses
Listings 7.10-7.11	242-246 (264-268)	Ollama query and scoring

Complete code files

File	Purpose
`config.py`	Tiny and all GPT-2 model configurations
`tokenizer.py`	Simple tokenizers, GPT-2 BPE access, sliding-window loader
`attention.py`	Simple, causal, wrapper, and efficient multi-head attention
`model.py`	LayerNorm, GELU, feed-forward, block, GPT, greedy generation
`train.py`	LM loss, pretraining loop, sampling, checkpoints, GPT-2 mapping
`finetune.py`	Spam and instruction datasets, collators, training utilities
`eval.py`	Classifier inference, response extraction, local model-based scoring

These modules preserve the book's essential operations and ordering. Small adaptations are intentionally visible: validation checks, device-safe tensor creation, batch-safe EOS stopping, and the corrected context axis above.

GitHub portfolio integration

Recommended repository structure

```
llm-from-scratch-2week/
+-- README.md
+-- BLOG_POST_TEMPLATE.md
+-- config.py
+-- tokenizer.py
+-- attention.py
+-- model.py
+-- train.py
+-- finetune.py
+-- eval.py
+-- notebooks/              # your 14 daily experiments
+-- artifacts/              # small plots and JSON samples, not huge weights
`-- tests/test_smoke.py     # shape, mask, round-trip, and collate tests
```

Do not commit the book PDF, downloaded GPT-2 weights, private data, or giant checkpoints. Add them to `.gitignore`. Check the upstream code license before publishing unchanged source and clearly attribute the book in your README.

What the public README should show

Lead with evidence, not a learning diary:

1. A one-paragraph problem and scope statement.
2. A compact architecture diagram: tokenization -> embeddings -> 12 decoder

blocks -> vocabulary/classification head.

3. A tensor-shape table and parameter count.
4. A loss plot with configuration, dataset split, seed, and runtime.
5. A decoding comparison with fixed prompt and settings.
6. Spam train/validation/test metrics and several predictions.
7. Instruction examples showing prompt, reference, response, and your rubric.
8. "What I would change at scale": better corpus, modern tokenizer, RoPE,

mixed precision, distributed training, LoRA, and stronger evaluation.

Use this results table and replace every placeholder:

Experiment	Model	Data	Metric	Result	Hardware/time
Tiny pretraining	4-layer GPT	The Verdict	validation loss	TBD	TBD
GPT-2 load check	124M	fixed prompt	coherent continuation	TBD	TBD
Spam fine-tuning	GPT-2 124M	SMS split	test accuracy	TBD	TBD
Instruction SFT	GPT-2 124M/355M	1,100 examples	manual mean / loss	TBD	TBD

Suggested commit sequence

1. `feat: add tokenizer and next-token dataset`
2. `feat: implement causal multi-head attention`
3. `feat: assemble decoder-only gpt model`
4. `feat: add pretraining and decoding strategies`

5. `feat: map pretrained gpt2 weights`
6. `feat: add spam classification fine-tuning`
7. `feat: add instruction tuning and evaluation`
8. `docs: publish results, limitations, and blog post`

Blog post plan

Use `BLOG_POST_TEMPLATE.md`. The strongest narrative is:

> *I rebuilt the GPT data path, attention, model, training loop, and two fine-tuning workflows. The important lesson was not that a laptop-trained model competes with production LLMs; it was learning exactly where token IDs become representations, where causal information flow is enforced, and how the same pretrained backbone becomes a generator or classifier.*

Include one code excerpt (causal mask), one diagram, one loss chart, one failure case, one fine-tuning comparison, and a candid limitations section.

Final success checklist

- [ ] A tokenizer encodes/decodes text and BPE handles unseen words.
- [ ] Sliding windows produce correctly shifted targets.
- [ ] Multi-head causal attention passes mask and shape assertions.
- [ ] A from-scratch GPT returns logits and generates tokens.
- [ ] A tiny model trains on unlabeled text with tracked validation loss.
- [ ] OpenAI GPT-2 weights load and produce coherent text.
- [ ] A spam classifier reports held-out accuracy.
- [ ] An instruction-tuned model produces saved held-out responses.
- [ ] Evaluation includes decoding settings, seed, rubric, and limitations.
- [ ] GitHub documentation and the blog post explain both results and failure modes.

Where this fits in your six-month roadmap

This project gives you the internals needed to reason about the rest of the roadmap:

- Prompt engineering: you will understand that prompts select a conditional distribution; they do not update weights.
- RAG: you will recognize retrieved context as additional tokens competing for a finite context window and attention budget.
- Agents: you will separate the LLM's token-generation role from external state, tools, planning loops, and evaluation.
- ML systems: you will have concrete artifacts for data versioning, checkpointing, latency, memory, monitoring, and reproducibility discussions.

Appendix - Complete Source Files

These seven modules are the runnable reference implementation used by the tutorial. They preserve the book's essential operations while adding explicit validation and the documented context-axis correction.

config.py

Complete listing - config.py

```python
"""Model configurations used throughout the accelerated tutorial.

The key names and GPT-2 dimensions follow the book.  Use TINY_CONFIG for
smoke tests, GPT_CONFIG_124M_TRAIN for the small "The Verdict" pretraining
exercise, and a full context configuration when loading OpenAI weights.
"""

GPT_CONFIG_124M = {
    "vocab_size": 50257,
    "context_length": 1024,
    "emb_dim": 768,
    "n_heads": 12,
    "n_layers": 12,
    "drop_rate": 0.1,
    "qkv_bias": False,
}

GPT_CONFIG_124M_TRAIN = {
    **GPT_CONFIG_124M,
    "context_length": 256,
}

TINY_CONFIG = {
    "vocab_size": 50257,
    "context_length": 64,
    "emb_dim": 128,
    "n_heads": 4,
    "n_layers": 4,
    "drop_rate": 0.1,
    "qkv_bias": False,
}

BASE_CONFIG = {
    "vocab_size": 50257,
    "context_length": 1024,
    "drop_rate": 0.0,
    "qkv_bias": True,
}

MODEL_CONFIGS = {
    "gpt2-small (124M)": {
        "emb_dim": 768,
        "n_layers": 12,
        "n_heads": 12,
    },
    "gpt2-medium (355M)": {
        "emb_dim": 1024,
        "n_layers": 24,
        "n_heads": 16,
    },
    "gpt2-large (774M)": {
        "emb_dim": 1280,
        "n_layers": 36,
        "n_heads": 20,
    },
```

```python
    "gpt2-xl (1558M)": {
        "emb_dim": 1600,
        "n_layers": 48,
        "n_heads": 25,
    },
}


def pretrained_config(model_name="gpt2-small (124M)"):
    """Return a fresh configuration compatible with OpenAI GPT-2 weights."""
    if model_name not in MODEL_CONFIGS:
        choices = ", ".join(MODEL_CONFIGS)
        raise ValueError(f"Unknown model {model_name!r}. Choose one of: {choices}")
    return {**BASE_CONFIG, **MODEL_CONFIGS[model_name]}
```

tokenizer.py

Complete listing - tokenizer.py

PYTHON

```
"""Text preparation code adapted from chapter 2 listings 2.1-2.6."""

import re
import urllib.request
from pathlib import Path

import tiktoken
import torch
from torch.utils.data import DataLoader, Dataset


VERDICT_URL = (
    "https://raw.githubusercontent.com/rasbt/"
    "LLMs-from-scratch/main/ch02/01_main-chapter-code/the-verdict.txt"
)


def download_verdict(file_path="the-verdict.txt"):
    """Download the public-domain practice text if it is not already local."""
    destination = Path(file_path)
    if not destination.exists():
        urllib.request.urlretrieve(VERDICT_URL, destination)
    return destination


def read_text(file_path):
    """Listing 2.1: read a UTF-8 text file."""
    with open(file_path, "r", encoding="utf-8") as file:
        return file.read()


def pretokenize(text):
    """Split words and punctuation using the chapter 2 teaching regex."""
    pieces = re.split(r'([,.:;?_!"()\']|--|\s)', text)
    return [piece.strip() for piece in pieces if piece.strip()]


def build_vocabulary(text, add_special_tokens=True):
    """Listing 2.2 plus the chapter 2 special tokens."""
    all_tokens = sorted(set(pretokenize(text)))
    if add_special_tokens:
        all_tokens.extend(["<|endoftext|>", "<|unk|>"])
    return {token: integer for integer, token in enumerate(all_tokens)}


class SimpleTokenizerV1:
    """Listing 2.3: a deliberately small word-and-punctuation tokenizer."""

    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {integer: string for string, integer in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.?_!"()\']|--|\s)', text)
        preprocessed = [
```

PYTHON (CONTINUED)

```
            item.strip() for item in preprocessed if item.strip()
        ]
        return [self.str_to_int[token] for token in preprocessed]

    def decode(self, ids):
        text = " ".join([self.int_to_str[token_id] for token_id in ids])
        return re.sub(r'\s+([,.?!"()\'])', r"\1", text)


class SimpleTokenizerV2:
    """Listing 2.4: the simple tokenizer with unknown-token handling."""

    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {integer: string for string, integer in vocab.items()}

    def encode(self, text):
        preprocessed = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        preprocessed = [
            item.strip() for item in preprocessed if item.strip()
        ]
        preprocessed = [
            item if item in self.str_to_int else "<|unk|>"
```

```
            for item in preprocessed
        ]
        return [self.str_to_int[token] for token in preprocessed]

    def decode(self, ids):
        text = " ".join([self.int_to_str[token_id] for token_id in ids])
        return re.sub(r'\s+([,.:;?!"()\'])', r"\1", text)


def get_gpt2_tokenizer():
    """Return the 50,257-token GPT-2 BPE tokenizer used by the model."""
    return tiktoken.get_encoding("gpt2")


class GPTDatasetV1(Dataset):
    """Listing 2.5: next-token examples created with a sliding window."""

    def __init__(self, txt, tokenizer, max_length, stride):
        self.input_ids = []
        self.target_ids = []
        token_ids = tokenizer.encode(txt)

        for i in range(0, len(token_ids) - max_length, stride):
            input_chunk = token_ids[i : i + max_length]
            target_chunk = token_ids[i + 1 : i + max_length + 1]
            self.input_ids.append(torch.tensor(input_chunk))
            self.target_ids.append(torch.tensor(target_chunk))

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
```

PYTHON (CONTINUED)

```
        return self.input_ids[idx], self.target_ids[idx]


def create_dataloader_v1(
    txt,
    batch_size=4,
    max_length=256,
    stride=128,
    shuffle=True,
    drop_last=True,
    num_workers=0,
):
    """Listing 2.6: construct batches of shifted input/target sequences."""
    tokenizer = get_gpt2_tokenizer()
    dataset = GPTDatasetV1(txt, tokenizer, max_length, stride)
    return DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        drop_last=drop_last,
        num_workers=num_workers,
    )
```

attention.py

Complete listing - attention.py

PYTHON

```
"""Self-attention implementations adapted from chapter 3 listings 3.1-3.5."""

import torch
import torch.nn as nn


class SelfAttentionV1(nn.Module):
    """Listing 3.1: trainable matrices stored directly as Parameters."""

    def __init__(self, d_in, d_out):
        super().__init__()
        self.W_query = nn.Parameter(torch.rand(d_in, d_out))
        self.W_key = nn.Parameter(torch.rand(d_in, d_out))
        self.W_value = nn.Parameter(torch.rand(d_in, d_out))

    def forward(self, x):
        keys = x @ self.W_key
        queries = x @ self.W_query
        values = x @ self.W_value
        attn_scores = queries @ keys.T
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1] ** 0.5, dim=-1
        )
        return attn_weights @ values


class SelfAttentionV2(nn.Module):
    """Listing 3.2: the same operation using optimized Linear layers."""

    def __init__(self, d_in, d_out, qkv_bias=False):
        super().__init__()
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)

    def forward(self, x):
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)
        attn_scores = queries @ keys.T
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1] ** 0.5, dim=-1
        )
        return attn_weights @ values


# Exact listing spellings retained for readers comparing line by line.
SelfAttention_v1 = SelfAttentionV1
SelfAttention_v2 = SelfAttentionV2


class CausalAttention(nn.Module):
    """Listing 3.3: batched single-head attention with causal masking."""

    def __init__(self, d_in, d_out, context_length, dropout, qkv_bias=False):
```

PYTHON (CONTINUED)

```
        super().__init__()
        self.d_out = d_out
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1),
        )

    def forward(self, x):
        _, num_tokens, _ = x.shape
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)
        attn_scores = queries @ keys.transpose(1, 2)
        attn_scores.masked_fill_(
            self.mask.bool()[:num_tokens, :num_tokens], -torch.inf
        )
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1] ** 0.5, dim=-1
        )
```

```
        attn_weights = self.dropout(attn_weights)
        return attn_weights @ values


class MultiHeadAttentionWrapper(nn.Module):
    """Listing 3.4: an intuitive, less efficient stack of attention heads."""

    def __init__(
        self,
        d_in,
        d_out,
        context_length,
        dropout,
        num_heads,
        qkv_bias=False,
    ):
        super().__init__()
        self.heads = nn.ModuleList(
            [
                CausalAttention(
                    d_in, d_out, context_length, dropout, qkv_bias
                )
                for _ in range(num_heads)
            ]
        )

    def forward(self, x):
        return torch.cat([head(x) for head in self.heads], dim=-1)


class MultiHeadAttention(nn.Module):
    """Listing 3.5: parallel causal multi-head attention with split weights."""
```

PYTHON (CONTINUED)

```
    def __init__(
        self,
        d_in,
        d_out,
        context_length,
        dropout,
        num_heads,
        qkv_bias=False,
    ):
        super().__init__()
        if d_out % num_heads != 0:
            raise ValueError("d_out must be divisible by num_heads")
        self.d_out = d_out
        self.num_heads = num_heads
        self.head_dim = d_out // num_heads
        self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_key = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.W_value = nn.Linear(d_in, d_out, bias=qkv_bias)
        self.out_proj = nn.Linear(d_out, d_out)
        self.dropout = nn.Dropout(dropout)
        self.register_buffer(
            "mask",
            torch.triu(torch.ones(context_length, context_length), diagonal=1),
        )

    def forward(self, x):
        batch_size, num_tokens, _ = x.shape
        keys = self.W_key(x)
        queries = self.W_query(x)
        values = self.W_value(x)

        keys = keys.view(
            batch_size, num_tokens, self.num_heads, self.head_dim
        )
        values = values.view(
            batch_size, num_tokens, self.num_heads, self.head_dim
        )
        queries = queries.view(
            batch_size, num_tokens, self.num_heads, self.head_dim
        )

        keys = keys.transpose(1, 2)
        queries = queries.transpose(1, 2)
        values = values.transpose(1, 2)

        attn_scores = queries @ keys.transpose(2, 3)
        mask_bool = self.mask.bool()[:num_tokens, :num_tokens]
        attn_scores.masked_fill_(mask_bool, -torch.inf)
        attn_weights = torch.softmax(
            attn_scores / keys.shape[-1] ** 0.5, dim=-1
        )
        attn_weights = self.dropout(attn_weights)

        context_vec = (attn_weights @ values).transpose(1, 2)
```

PYTHON (CONTINUED)

```
        context_vec = context_vec.contiguous().view(
            batch_size, num_tokens, self.d_out
        )
        return self.out_proj(context_vec)
```

model.py

Complete listing - model.py

PYTHON

```
"""GPT-2-style model components adapted from chapter 4 listings 4.2-4.8."""

import torch
import torch.nn as nn

from attention import MultiHeadAttention


class LayerNorm(nn.Module):
    """Listing 4.2: normalize across each token's embedding dimension."""

    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale * norm_x + self.shift


class GELU(nn.Module):
    """Listing 4.3: the tanh approximation used by the original GPT-2."""

    def __init__(self):
        super().__init__()

    def forward(self, x):
        return 0.5 * x * (
            1
            + torch.tanh(
                torch.sqrt(torch.tensor(2.0 / torch.pi, device=x.device))
                * (x + 0.044715 * torch.pow(x, 3))
            )
        )


class FeedForward(nn.Module):
    """Listing 4.4: expand 4x, apply GELU, then project back."""

    def __init__(self, cfg):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
            GELU(),
            nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
        )

    def forward(self, x):
        return self.layers(x)
```

PYTHON (CONTINUED)

```
class ExampleDeepNeuralNetwork(nn.Module):
    """Listing 4.5: illustrate gradient flow with optional shortcuts."""

    def __init__(self, layer_sizes, use_shortcut):
        super().__init__()
        self.use_shortcut = use_shortcut
        self.layers = nn.ModuleList(
            [
                nn.Sequential(nn.Linear(layer_sizes[0], layer_sizes[1]), GELU()),
                nn.Sequential(nn.Linear(layer_sizes[1], layer_sizes[2]), GELU()),
                nn.Sequential(nn.Linear(layer_sizes[2], layer_sizes[3]), GELU()),
                nn.Sequential(nn.Linear(layer_sizes[3], layer_sizes[4]), GELU()),
                nn.Sequential(nn.Linear(layer_sizes[4], layer_sizes[5]), GELU()),
            ]
        )

    def forward(self, x):
        for layer in self.layers:
            layer_output = layer(x)
            if self.use_shortcut and x.shape == layer_output.shape:
                x = x + layer_output
            else:
                x = layer_output
        return x
```

```
class TransformerBlock(nn.Module):
    """Listing 4.6: pre-norm attention and MLP with residual paths."""

    def __init__(self, cfg):
        super().__init__()
        self.att = MultiHeadAttention(
            d_in=cfg["emb_dim"],
            d_out=cfg["emb_dim"],
            context_length=cfg["context_length"],
            num_heads=cfg["n_heads"],
            dropout=cfg["drop_rate"],
            qkv_bias=cfg["qkv_bias"],
        )
        self.ff = FeedForward(cfg)
        self.norm1 = LayerNorm(cfg["emb_dim"])
        self.norm2 = LayerNorm(cfg["emb_dim"])
        self.drop_shortcut = nn.Dropout(cfg["drop_rate"])

    def forward(self, x):
        shortcut = x
        x = self.norm1(x)
        x = self.att(x)
        x = self.drop_shortcut(x)
        x = x + shortcut

        shortcut = x
        x = self.norm2(x)
        x = self.ff(x)
        x = self.drop_shortcut(x)
```

PYTHON (CONTINUED)

```
        return x + shortcut


class GPTModel(nn.Module):
    """Listing 4.7: decoder-only GPT model."""

    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.Sequential(
            *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])]
        )
        self.final_norm = LayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        _, seq_len = in_idx.shape
        if seq_len > self.pos_emb.num_embeddings:
            raise ValueError(
                f"Sequence length {seq_len} exceeds model context "
                f"{self.pos_emb.num_embeddings}"
            )
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(
            torch.arange(seq_len, device=in_idx.device)
        )
        x = self.drop_emb(tok_embeds + pos_embeds)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        return self.out_head(x)


def generate_text_simple(model, idx, max_new_tokens, context_size):
    """Listing 4.8: deterministic greedy decoding."""
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        logits = logits[:, -1, :]
        idx_next = torch.argmax(logits, dim=-1, keepdim=True)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx


def count_parameters(model, trainable_only=False):
    """Return the number of parameters, optionally excluding frozen ones."""
    parameters = model.parameters()
    if trainable_only:
        parameters = (p for p in parameters if p.requires_grad)
    return sum(parameter.numel() for parameter in parameters)
```

train.py

Complete listing - train.py

PYTHON

```
"""Pretraining, decoding, checkpoints, and GPT-2 weight loading (chapter 5)."""

import importlib.util
import urllib.request
from pathlib import Path

import numpy as np
import torch
import torch.nn.functional as F

from config import pretrained_config
from model import GPTModel, generate_text_simple


GPT_DOWNLOAD_URL = (
    "https://raw.githubusercontent.com/rasbt/"
    "LLMs-from-scratch/main/ch05/01_main-chapter-code/gpt_download.py"
)


def text_to_token_ids(text, tokenizer):
    """Listing 5.1: encode text and add a batch dimension."""
    encoded = tokenizer.encode(
        text, allowed_special={"<|endoftext|>"}
    )
    return torch.tensor(encoded).unsqueeze(0)


def token_ids_to_text(token_ids, tokenizer):
    """Listing 5.1: remove the batch dimension and decode."""
    flat = token_ids.squeeze(0)
    return tokenizer.decode(flat.tolist())


def calc_loss_batch(input_batch, target_batch, model, device):
    """Cross-entropy next-token loss for one language-model batch."""
    input_batch = input_batch.to(device)
    target_batch = target_batch.to(device)
    logits = model(input_batch)
    return F.cross_entropy(
        logits.flatten(0, 1), target_batch.flatten()
    )


def calc_loss_loader(data_loader, model, device, num_batches=None):
    """Listing 5.2: mean language-model loss over a data loader."""
    if len(data_loader) == 0:
        return float("nan")
    if num_batches is None:
        num_batches = len(data_loader)
    else:
        num_batches = min(num_batches, len(data_loader))
    if num_batches == 0:
        return float("nan")
```

PYTHON (CONTINUED)

```
    total_loss = 0.0
    for i, (input_batch, target_batch) in enumerate(data_loader):
        if i >= num_batches:
            break
        loss = calc_loss_batch(
            input_batch, target_batch, model, device
        )
        total_loss += loss.item()
    return total_loss / num_batches


def evaluate_model(model, train_loader, val_loader, device, eval_iter):
    """Evaluate stable train/validation losses with dropout disabled."""
    model.eval()
    with torch.no_grad():
        train_loss = calc_loss_loader(
            train_loader, model, device, num_batches=eval_iter
        )
        val_loss = calc_loss_loader(
            val_loader, model, device, num_batches=eval_iter
        )
    model.train()
    return train_loss, val_loss
```

```
def generate_and_print_sample(model, tokenizer, device, start_context):
    """Generate a greedy progress sample without changing training state."""
    model.eval()
    context_size = model.pos_emb.weight.shape[0]
    encoded = text_to_token_ids(start_context, tokenizer).to(device)
    with torch.no_grad():
        token_ids = generate_text_simple(
            model=model,
            idx=encoded,
            max_new_tokens=50,
            context_size=context_size,
        )
    decoded_text = token_ids_to_text(token_ids, tokenizer)
    print(decoded_text.replace("\n", " "))
    model.train()


def train_model_simple(
    model,
    train_loader,
    val_loader,
    optimizer,
    device,
    num_epochs,
    eval_freq,
    eval_iter,
    start_context,
    tokenizer,
):
    """Listing 5.3: the compact LLM pretraining/fine-tuning loop."""
```

PYTHON (CONTINUED)

```
    train_losses, val_losses, track_tokens_seen = [], [], []
    tokens_seen, global_step = 0, -1

    for epoch in range(num_epochs):
        model.train()
        for input_batch, target_batch in train_loader:
            optimizer.zero_grad()
            loss = calc_loss_batch(
                input_batch, target_batch, model, device
            )
            loss.backward()
            optimizer.step()
            tokens_seen += input_batch.numel()
            global_step += 1

            if global_step % eval_freq == 0:
                train_loss, val_loss = evaluate_model(
                    model,
                    train_loader,
                    val_loader,
                    device,
                    eval_iter,
                )
                train_losses.append(train_loss)
                val_losses.append(val_loss)
                track_tokens_seen.append(tokens_seen)
                print(
                    f"Ep {epoch + 1} (Step {global_step:06d}): "
                    f"Train loss {train_loss:.3f}, "
                    f"Val loss {val_loss:.3f}"
                )

        generate_and_print_sample(
            model, tokenizer, device, start_context
        )

    return train_losses, val_losses, track_tokens_seen


def generate(
    model,
    idx,
    max_new_tokens,
    context_size,
    temperature=0.0,
    top_k=None,
    eos_id=None,
):
    """Listing 5.4: decoding with temperature, top-k, and EOS stopping."""
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        logits = logits[:, -1, :]
```

PYTHON (CONTINUED)

```
        if top_k is not None:
            if top_k <= 0:
                raise ValueError("top_k must be a positive integer")
            top_logits, _ = torch.topk(
                logits, min(top_k, logits.shape[-1])
            )
            min_val = top_logits[:, -1, None]
            logits = torch.where(
                logits < min_val,
                torch.tensor(float("-inf"), device=logits.device),
                logits,
            )

        if temperature > 0.0:
            logits = logits / temperature
            probs = torch.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
        else:
            idx_next = torch.argmax(logits, dim=-1, keepdim=True)

        idx = torch.cat((idx, idx_next), dim=1)
        if eos_id is not None and torch.all(idx_next == eos_id):
            break
    return idx


def perplexity(loss):
    """Convert average token cross-entropy to perplexity."""
    return torch.exp(torch.as_tensor(loss)).item()


def save_checkpoint(path, model, optimizer=None, **metadata):
    """Save model weights and, optionally, optimizer state for resuming."""
    checkpoint = {"model_state_dict": model.state_dict(), **metadata}
    if optimizer is not None:
        checkpoint["optimizer_state_dict"] = optimizer.state_dict()
    torch.save(checkpoint, path)


def load_checkpoint(path, model, optimizer=None, device="cpu"):
    """Load a checkpoint created by save_checkpoint."""
    checkpoint = torch.load(path, map_location=device)
    model.load_state_dict(checkpoint["model_state_dict"])
    if optimizer is not None and "optimizer_state_dict" in checkpoint:
        optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
    return checkpoint


def assign(left, right):
    """Validate a source array and turn it into a trainable Parameter."""
    if left.shape != right.shape:
        raise ValueError(
            f"Shape mismatch. Left: {left.shape}, Right: {right.shape}"
        )
    tensor = torch.as_tensor(
```

PYTHON (CONTINUED)

```
        right, dtype=left.dtype, device=left.device
    ).clone()
    return torch.nn.Parameter(tensor)


def load_weights_into_gpt(gpt, params):
    """Listing 5.5: map OpenAI's TensorFlow GPT-2 arrays into GPTModel."""
    gpt.pos_emb.weight = assign(gpt.pos_emb.weight, params["wpe"])
    gpt.tok_emb.weight = assign(gpt.tok_emb.weight, params["wte"])

    for block_index, block_params in enumerate(params["blocks"]):
        block = gpt.trf_blocks[block_index]
        q_w, k_w, v_w = np.split(
            block_params["attn"]["c_attn"]["w"], 3, axis=-1
        )
        block.att.W_query.weight = assign(
            block.att.W_query.weight, q_w.T
        )
        block.att.W_key.weight = assign(
            block.att.W_key.weight, k_w.T
        )
        block.att.W_value.weight = assign(
            block.att.W_value.weight, v_w.T
        )

        q_b, k_b, v_b = np.split(
            block_params["attn"]["c_attn"]["b"], 3, axis=-1
        )
        block.att.W_query.bias = assign(block.att.W_query.bias, q_b)
```

```
        block.att.W_key.bias = assign(block.att.W_key.bias, k_b)
        block.att.W_value.bias = assign(block.att.W_value.bias, v_b)

        block.att.out_proj.weight = assign(
            block.att.out_proj.weight,
            block_params["attn"]["c_proj"]["w"].T,
        )
        block.att.out_proj.bias = assign(
            block.att.out_proj.bias,
            block_params["attn"]["c_proj"]["b"],
        )
        block.ff.layers[0].weight = assign(
            block.ff.layers[0].weight,
            block_params["mlp"]["c_fc"]["w"].T,
        )
        block.ff.layers[0].bias = assign(
            block.ff.layers[0].bias,
            block_params["mlp"]["c_fc"]["b"],
        )
        block.ff.layers[2].weight = assign(
            block.ff.layers[2].weight,
            block_params["mlp"]["c_proj"]["w"].T,
        )
        block.ff.layers[2].bias = assign(
            block.ff.layers[2].bias,
            block_params["mlp"]["c_proj"]["b"],
```

PYTHON (CONTINUED)

```
        )
        block.norm1.scale = assign(
            block.norm1.scale, block_params["ln_1"]["g"]
        )
        block.norm1.shift = assign(
            block.norm1.shift, block_params["ln_1"]["b"]
        )
        block.norm2.scale = assign(
            block.norm2.scale, block_params["ln_2"]["g"]
        )
        block.norm2.shift = assign(
            block.norm2.shift, block_params["ln_2"]["b"]
        )

    gpt.final_norm.scale = assign(gpt.final_norm.scale, params["g"])
    gpt.final_norm.shift = assign(gpt.final_norm.shift, params["b"])
    gpt.out_head.weight = assign(gpt.out_head.weight, params["wte"])


def ensure_gpt_download_module(destination="gpt_download.py"):
    """Fetch the official helper referenced on book page 161 when absent."""
    destination = Path(destination)
    if not destination.exists():
        urllib.request.urlretrieve(GPT_DOWNLOAD_URL, destination)
    return destination


def _load_download_module(module_path):
    spec = importlib.util.spec_from_file_location(
        "gpt_download", str(module_path)
    )
    if spec is None or spec.loader is None:
        raise ImportError(f"Could not import {module_path}")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module


def load_pretrained_gpt2(
    model_name="gpt2-small (124M)",
    models_dir="gpt2",
    download_module_path="gpt_download.py",
):
    """Download official GPT-2 weights, map them, and return model/settings.

    The official helper reads TensorFlow checkpoints, so install TensorFlow and
    tqdm for this one exercise.  The rest of the project needs only PyTorch,
    tiktoken, NumPy, and pandas.
    """
    cfg = pretrained_config(model_name)
    model_size = model_name.split(" ")[-1].lstrip("(").rstrip(")")
    module_path = ensure_gpt_download_module(download_module_path)
    module = _load_download_module(module_path)
    settings, params = module.download_and_load_gpt2(
        model_size=model_size, models_dir=models_dir
```

PYTHON (CONTINUED)

```
    )
    model = GPTModel(cfg)
```

```
    load_weights_into_gpt(model, params)
    model.eval()
    return model, cfg, settings
```

finetune.py

Complete listing - finetune.py

```python
"""Classification and instruction fine-tuning utilities (chapters 6 and 7)."""

import json
import os
import urllib.request
import zipfile
from functools import partial
from pathlib import Path

import pandas as pd
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader, Dataset


SPAM_URL = (
    "https://archive.ics.uci.edu/static/public/228/"
    "sms+spam+collection.zip"
)
INSTRUCTION_URL = (
    "https://raw.githubusercontent.com/rasbt/LLMs-from-scratch/"
    "main/ch07/01_main-chapter-code/instruction-data.json"
)


def download_and_unzip_spam_data(
    url=SPAM_URL,
    zip_path="sms_spam_collection.zip",
    extracted_path="sms_spam_collection",
    data_file_path=None,
):
    """Listing 6.1: download and extract the SMS spam collection."""
    if data_file_path is None:
        data_file_path = Path(extracted_path) / "SMSSpamCollection.tsv"
    data_file_path = Path(data_file_path)
    if data_file_path.exists():
        return data_file_path

    with urllib.request.urlopen(url) as response:
        with open(zip_path, "wb") as out_file:
            out_file.write(response.read())
    with zipfile.ZipFile(zip_path, "r") as zip_ref:
        zip_ref.extractall(extracted_path)
    original = Path(extracted_path) / "SMSSpamCollection"
    os.rename(original, data_file_path)
    return data_file_path


def create_balanced_dataset(dataframe):
    """Listing 6.2: undersample ham messages to match the spam count."""
    num_spam = dataframe[dataframe["Label"] == "spam"].shape[0]
    ham_subset = dataframe[dataframe["Label"] == "ham"].sample(
        num_spam, random_state=123
    )
    return pd.concat(
```

```python
        [ham_subset, dataframe[dataframe["Label"] == "spam"]]
    )


def random_split(dataframe, train_frac=0.7, validation_frac=0.1):
    """Listing 6.3: deterministic train/validation/test split."""
    dataframe = dataframe.sample(
        frac=1, random_state=123
    ).reset_index(drop=True)
    train_end = int(len(dataframe) * train_frac)
    validation_end = train_end + int(
        len(dataframe) * validation_frac
    )
    return (
        dataframe[:train_end],
        dataframe[train_end:validation_end],
        dataframe[validation_end:],
    )


def prepare_spam_csvs(data_file_path, output_dir="."):
    """Balance, encode labels, split, and save the chapter 6 CSV files."""
    dataframe = pd.read_csv(
```

```
        data_file_path,
        sep="\t",
        header=None,
        names=["Label", "Text"],
    )
    balanced = create_balanced_dataset(dataframe)
    balanced["Label"] = balanced["Label"].map({"ham": 0, "spam": 1})
    train_df, val_df, test_df = random_split(balanced)
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    paths = {
        "train": output_dir / "train.csv",
        "validation": output_dir / "validation.csv",
        "test": output_dir / "test.csv",
    }
    train_df.to_csv(paths["train"], index=None)
    val_df.to_csv(paths["validation"], index=None)
    test_df.to_csv(paths["test"], index=None)
    return paths


class SpamDataset(Dataset):
    """Listing 6.4: pretokenized, padded SMS classification dataset."""

    def __init__(
        self,
        csv_file,
        tokenizer,
        max_length=None,
        pad_token_id=50256,
    ):
        self.data = pd.read_csv(csv_file)
```

PYTHON (CONTINUED)

```
        self.encoded_texts = [
            tokenizer.encode(text) for text in self.data["Text"]
        ]
        if max_length is None:
            self.max_length = self._longest_encoded_length()
        else:
            self.max_length = max_length
            self.encoded_texts = [
                encoded_text[: self.max_length]
                for encoded_text in self.encoded_texts
            ]
        self.encoded_texts = [
            encoded_text
            + [pad_token_id] * (self.max_length - len(encoded_text))
            for encoded_text in self.encoded_texts
        ]

    def __getitem__(self, index):
        encoded = self.encoded_texts[index]
        label = self.data.iloc[index]["Label"]
        return (
            torch.tensor(encoded, dtype=torch.long),
            torch.tensor(label, dtype=torch.long),
        )

    def __len__(self):
        return len(self.data)

    def _longest_encoded_length(self):
        return max(len(encoded) for encoded in self.encoded_texts)


def create_spam_dataloaders(
    train_csv,
    validation_csv,
    test_csv,
    tokenizer,
    batch_size=8,
    num_workers=0,
):
    """Listing 6.5: create consistently padded classification loaders."""
    train_dataset = SpamDataset(train_csv, tokenizer)
    val_dataset = SpamDataset(
        validation_csv,
        tokenizer,
        max_length=train_dataset.max_length,
    )
    test_dataset = SpamDataset(
        test_csv,
        tokenizer,
        max_length=train_dataset.max_length,
    )
    train_loader = DataLoader(
        train_dataset,
```

```
        batch_size=batch_size,
```

PYTHON (CONTINUED)

```
        shuffle=True,
        num_workers=num_workers,
        drop_last=True,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        num_workers=num_workers,
        drop_last=False,
    )
    test_loader = DataLoader(
        test_dataset,
        batch_size=batch_size,
        num_workers=num_workers,
        drop_last=False,
    )
    return train_loader, val_loader, test_loader, train_dataset.max_length


def configure_classifier(model, emb_dim, num_classes=2):
    """Listing 6.7: freeze GPT, add a head, then unfreeze the last block."""
    for parameter in model.parameters():
        parameter.requires_grad = False
    model.out_head = torch.nn.Linear(emb_dim, num_classes)
    for parameter in model.trf_blocks[-1].parameters():
        parameter.requires_grad = True
    for parameter in model.final_norm.parameters():
        parameter.requires_grad = True
    return model


def calc_accuracy_loader(data_loader, model, device, num_batches=None):
    """Listing 6.8: last-token classification accuracy."""
    model.eval()
    correct_predictions, num_examples = 0, 0
    if num_batches is None:
        num_batches = len(data_loader)
    else:
        num_batches = min(num_batches, len(data_loader))

    for i, (input_batch, target_batch) in enumerate(data_loader):
        if i >= num_batches:
            break
        input_batch = input_batch.to(device)
        target_batch = target_batch.to(device)
        with torch.no_grad():
            logits = model(input_batch)[:, -1, :]
        predicted_labels = torch.argmax(logits, dim=-1)
        num_examples += predicted_labels.shape[0]
        correct_predictions += (
            predicted_labels == target_batch
        ).sum().item()
    return correct_predictions / num_examples if num_examples else float("nan")
```

PYTHON (CONTINUED)

```
def calc_classification_loss_batch(
    input_batch, target_batch, model, device
):
    """Cross-entropy loss on the final output token."""
    input_batch = input_batch.to(device)
    target_batch = target_batch.to(device)
    logits = model(input_batch)[:, -1, :]
    return F.cross_entropy(logits, target_batch)


def calc_classification_loss_loader(
    data_loader, model, device, num_batches=None
):
    """Listing 6.9: mean classification loss over a loader."""
    if len(data_loader) == 0:
        return float("nan")
    if num_batches is None:
        num_batches = len(data_loader)
    else:
        num_batches = min(num_batches, len(data_loader))
    if num_batches == 0:
        return float("nan")

    total_loss = 0.0
    for i, (input_batch, target_batch) in enumerate(data_loader):
        if i >= num_batches:
            break
        loss = calc_classification_loss_batch(
            input_batch, target_batch, model, device
```

```
        )
        total_loss += loss.item()
    return total_loss / num_batches


# Chapter 6 uses these shorter names. The explicit aliases avoid confusing
# them with the language-model loss functions in train.py when both are used.
calc_loss_batch = calc_classification_loss_batch
calc_loss_loader = calc_classification_loss_loader


def evaluate_classifier(model, train_loader, val_loader, device, eval_iter):
    """Evaluate classification loss while dropout and gradients are disabled."""
    model.eval()
    with torch.no_grad():
        train_loss = calc_classification_loss_loader(
            train_loader, model, device, num_batches=eval_iter
        )
        val_loss = calc_classification_loss_loader(
            val_loader, model, device, num_batches=eval_iter
        )
    model.train()
    return train_loss, val_loss


evaluate_model = evaluate_classifier
```

PYTHON (CONTINUED)

```
def train_classifier_simple(
    model,
    train_loader,
    val_loader,
    optimizer,
    device,
    num_epochs,
    eval_freq,
    eval_iter,
):
    """Listing 6.10: supervised classifier fine-tuning loop."""
    train_losses, val_losses, train_accs, val_accs = [], [], [], []
    examples_seen, global_step = 0, -1

    for epoch in range(num_epochs):
        model.train()
        for input_batch, target_batch in train_loader:
            optimizer.zero_grad()
            loss = calc_classification_loss_batch(
                input_batch, target_batch, model, device
            )
            loss.backward()
            optimizer.step()
            examples_seen += input_batch.shape[0]
            global_step += 1

            if global_step % eval_freq == 0:
                train_loss, val_loss = evaluate_classifier(
                    model,
                    train_loader,
                    val_loader,
                    device,
                    eval_iter,
                )
                train_losses.append(train_loss)
                val_losses.append(val_loss)
                print(
                    f"Ep {epoch + 1} (Step {global_step:06d}): "
                    f"Train loss {train_loss:.3f}, "
                    f"Val loss {val_loss:.3f}"
                )

        train_accuracy = calc_accuracy_loader(
            train_loader, model, device, num_batches=eval_iter
        )
        val_accuracy = calc_accuracy_loader(
            val_loader, model, device, num_batches=eval_iter
        )
        print(
            f"Training accuracy: {train_accuracy * 100:.2f}% | "
            f"Validation accuracy: {val_accuracy * 100:.2f}%"
        )
        train_accs.append(train_accuracy)
```

PYTHON (CONTINUED)

```
        val_accs.append(val_accuracy)

    return (
        train_losses,
```

```
        val_losses,
        train_accs,
        val_accs,
        examples_seen,
    )


def download_and_load_instruction_data(
    file_path="instruction-data.json", url=INSTRUCTION_URL
):
    """Listing 7.1: download and parse the book's instruction dataset."""
    file_path = Path(file_path)
    if not file_path.exists():
        with urllib.request.urlopen(url) as response:
            text_data = response.read().decode("utf-8")
        file_path.write_text(text_data, encoding="utf-8")
    return json.loads(file_path.read_text(encoding="utf-8"))


def format_input(entry):
    """Listing 7.2: format an entry with the Alpaca prompt template."""
    instruction_text = (
        "Below is an instruction that describes a task. "
        "Write a response that appropriately completes the request."
        f"\n\n### Instruction:\n{entry['instruction']}"
    )
    input_text = (
        f"\n\n### Input:\n{entry['input']}" if entry["input"] else ""
    )
    return instruction_text + input_text


def split_instruction_data(data):
    """Listing 7.3: 85% train, 5% validation, and 10% test."""
    train_portion = int(len(data) * 0.85)
    test_portion = int(len(data) * 0.1)
    train_data = data[:train_portion]
    test_data = data[train_portion : train_portion + test_portion]
    val_data = data[train_portion + test_portion :]
    return train_data, val_data, test_data


class InstructionDataset(Dataset):
    """Listing 7.4: pretokenize formatted instruction-response examples."""

    def __init__(self, data, tokenizer):
        self.data = data
        self.encoded_texts = []
        for entry in data:
            instruction_plus_input = format_input(entry)
            response_text = f"\n\n### Response:\n{entry['output']}"
```

PYTHON (CONTINUED)

```
            full_text = instruction_plus_input + response_text
            self.encoded_texts.append(tokenizer.encode(full_text))

    def __getitem__(self, index):
        return self.encoded_texts[index]

    def __len__(self):
        return len(self.data)


def custom_collate_fn(
    batch,
    pad_token_id=50256,
    ignore_index=-100,
    allowed_max_length=None,
    device="cpu",
):
    """Listing 7.5: dynamic padding and ignored padding targets."""
    batch_max_length = max(len(item) + 1 for item in batch)
    inputs_list, targets_list = [], []

    for item in batch:
        new_item = item.copy()
        new_item += [pad_token_id]
        padded = new_item + [pad_token_id] * (
            batch_max_length - len(new_item)
        )
        inputs = torch.tensor(padded[:-1])
        targets = torch.tensor(padded[1:])
        indices = torch.nonzero(
            targets == pad_token_id, as_tuple=False
        ).flatten()
        if indices.numel() > 1:
            targets[indices[1:]] = ignore_index
        if allowed_max_length is not None:
```

```
            inputs = inputs[:allowed_max_length]
            targets = targets[:allowed_max_length]
        inputs_list.append(inputs)
        targets_list.append(targets)

    inputs_tensor = torch.stack(inputs_list).to(device)
    targets_tensor = torch.stack(targets_list).to(device)
    return inputs_tensor, targets_tensor


def create_instruction_dataloaders(
    train_data,
    val_data,
    test_data,
    tokenizer,
    device="cpu",
    batch_size=8,
    allowed_max_length=1024,
    num_workers=0,
):
```

PYTHON (CONTINUED)

```
    """Listing 7.6: create dynamically padded instruction loaders."""
    collate = partial(
        custom_collate_fn,
        device=device,
        allowed_max_length=allowed_max_length,
    )
    train_dataset = InstructionDataset(train_data, tokenizer)
    val_dataset = InstructionDataset(val_data, tokenizer)
    test_dataset = InstructionDataset(test_data, tokenizer)
    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        collate_fn=collate,
        shuffle=True,
        drop_last=True,
        num_workers=num_workers,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        collate_fn=collate,
        shuffle=False,
        drop_last=False,
        num_workers=num_workers,
    )
    test_loader = DataLoader(
        test_dataset,
        batch_size=batch_size,
        collate_fn=collate,
        shuffle=False,
        drop_last=False,
        num_workers=num_workers,
    )
    return train_loader, val_loader, test_loader
```

eval.py

Complete listing - eval.py

PYTHON

```
"""Inference and evaluation helpers for chapters 6 and 7."""

import json
import urllib.request
from pathlib import Path

import torch

from finetune import format_input
from train import generate, text_to_token_ids, token_ids_to_text


def classify_text(
    text,
    model,
    tokenizer,
    device,
    max_length=None,
    pad_token_id=50256,
):
    """Listing 6.12, corrected to read the positional context axis.

    The PDF prints ``model.pos_emb.weight.shape[1]`` on book page 201.
    Positional embedding weights have shape ``[context_length, emb_dim]``,
    so the working implementation must use axis 0.
    """
    model.eval()
    input_ids = tokenizer.encode(text)
    supported_context_length = model.pos_emb.weight.shape[0]
    if max_length is None:
        raise ValueError(
            "max_length is required; use the training dataset's max_length"
        )
    max_length = min(max_length, supported_context_length)
    input_ids = input_ids[:max_length]
    input_ids += [pad_token_id] * (max_length - len(input_ids))
    input_tensor = torch.tensor(
        input_ids, device=device
    ).unsqueeze(0)

    with torch.no_grad():
        logits = model(input_tensor)[:, -1, :]
    predicted_label = torch.argmax(logits, dim=-1).item()
    return "spam" if predicted_label == 1 else "not spam"


def classify_review(
    text,
    model,
    tokenizer,
    device,
    max_length=None,
    pad_token_id=50256,
):
    """Exact Listing 6.12 function name, backed by the corrected helper."""
```

PYTHON (CONTINUED)

```
    return classify_text(
        text,
        model,
        tokenizer,
        device,
        max_length=max_length,
        pad_token_id=pad_token_id,
    )


def extract_response(generated_text, input_text):
    """Remove the original prompt and response marker from generated text."""
    return (
        generated_text[len(input_text) :]
        .replace("### Response:", "")
        .strip()
    )


def generate_test_responses(
    model,
    test_data,
    tokenizer,
```

```
    device,
    context_size,
    max_new_tokens=256,
    eos_id=50256,
):
    """Listing 7.9: add a generated response to each held-out example."""
    model.eval()
    results = []
    for entry in test_data:
        input_text = format_input(entry)
        token_ids = generate(
            model=model,
            idx=text_to_token_ids(input_text, tokenizer).to(device),
            max_new_tokens=max_new_tokens,
            context_size=context_size,
            eos_id=eos_id,
        )
        generated_text = token_ids_to_text(token_ids, tokenizer)
        enriched_entry = dict(entry)
        enriched_entry["model_response"] = extract_response(
            generated_text, input_text
        )
        results.append(enriched_entry)
    return results


def save_json(data, output_path):
    """Save human-readable evaluation artifacts."""
    output_path = Path(output_path)
    output_path.write_text(
        json.dumps(data, indent=4, ensure_ascii=False),
        encoding="utf-8",
```

PYTHON (CONTINUED)

```
    )


def query_model(
    prompt,
    model="llama3",
    url="http://localhost:11434/api/chat",
):
    """Listing 7.10: query a local Ollama evaluator through its REST API."""
    data = {
        "model": model,
        "messages": [{"role": "user", "content": prompt}],
        "options": {
            "seed": 123,
            "temperature": 0,
            "num_ctx": 2048,
        },
    }
    payload = json.dumps(data).encode("utf-8")
    request = urllib.request.Request(
        url, data=payload, method="POST"
    )
    request.add_header("Content-Type", "application/json")
    response_data = ""
    with urllib.request.urlopen(request) as response:
        while True:
            line = response.readline().decode("utf-8")
            if not line:
                break
            response_json = json.loads(line)
            response_data += response_json["message"]["content"]
    return response_data


def generate_model_scores(
    json_data,
    json_key="model_response",
    evaluator_model="llama3",
):
    """Listing 7.11: obtain 0-100 response-quality scores from Ollama."""
    scores = []
    for entry in json_data:
        prompt = (
            f"Given the input `{format_input(entry)}` "
            f"and correct output `{entry['output']}`, "
            f"score the model response `{entry[json_key]}` "
            "on a scale from 0 to 100, where 100 is the best score. "
            "Respond with the integer number only."
        )
        score = query_model(prompt, evaluator_model)
        try:
            scores.append(int(score.strip()))
        except ValueError:
            print(f"Could not convert score: {score}")
```

```
    return scores
```

PYTHON (CONTINUED)

```
def summarize_scores(scores):
    """Return count and mean for automated response scores."""
    if not scores:
        return {"count": 0, "mean": float("nan")}
    return {"count": len(scores), "mean": sum(scores) / len(scores)}
```